Conferencia 6: diccionarios y conjuntos

Programación · Ciencia de la Computación y Ciencia de Datos · MatCom · 2026-10-21

La clase pasada terminó con una pregunta que no pudimos responder: cuántas veces aparece cada palabra de un texto. Hoy la respondemos con el **diccionario**, la colección que asocia cada clave con un valor, y con su pariente cercano, el **conjunto**, que guarda elementos sin repetir. El proyecto de la clase es un programa que lee un archivo y reporta las diez palabras más frecuentes.

Trabajamos sobre dos archivos que tienes junto a estas notas: `quijote.txt`, los tres párrafos de la clase pasada, y `quijote-cap1.txt`, el primer capítulo completo del Quijote, tomado de la edición del Proyecto Gutenberg. El primero es pequeño para poder seguirlo a mano; el segundo es lo bastante grande para que las diferencias de velocidad se vean.

1. El problema, con lo que ya sabemos

Empecemos por lo que sí sabemos hacer. Con dos listas en paralelo, una de palabras vistas y otra de conteos, la palabra de la posición `i` en la primera tiene su conteo en la posición `i` de la segunda. Las funciones `limpiar` y `palabras_de` son las de la clase pasada:

```
SIGNOS = ".,;:¿?¡!()«»\"'—-"

def limpiar(texto):
    """Devuelve el texto en minúsculas y sin signos de puntuación."""
    texto = texto.lower()

    for signo in SIGNOS:
        texto = texto.replace(signo, " ")

    return texto

def palabras_de(ruta):
    """Devuelve la lista de palabras de un archivo de texto."""
    with open(ruta, encoding="utf-8") as archivo:
        return limpiar(archivo.read()).split()

def frecuencias_con_listas(palabras):
    """Devuelve dos listas paralelas: las palabras distintas y sus conteos."""
    vistas = []
    conteos = []

    for palabra in palabras:
        if palabra in vistas:
            i = vistas.index(palabra)
            conteos[i] += 1
        else:
            vistas.append(palabra)
            conteos.append(1)

    return vistas, conteos

vistas, conteos = frecuencias_con_listas(palabras_de("quijote.txt"))

print(len(vistas), "palabras distintas")
print(vistas[:6])
print(conteos[:6])
```

```
126 palabras distintas
['en', 'un', 'lugar', 'de', 'la', 'mancha']
[5, 3, 1, 20, 4, 1]
```

Funciona. El método `index` de una lista devuelve la posición de un elemento, y la tupla que devuelve la función se desempaqueta en dos nombres, como vimos con las listas.

Pero mira lo que hace cada vuelta del ciclo. `palabra in vistas` recorre la lista de palabras vistas de una en una hasta encontrarla, y después `index` la recorre otra vez para saber dónde estaba. Cada palabra del texto paga un recorrido de todas las palabras distintas encontradas hasta ese momento. Y el programa es incómodo de leer, porque la relación entre una palabra y su conteo no está escrita en ningún sitio: solo la sostiene la coincidencia de posiciones.

Lo que de verdad queremos es una tabla donde se pueda preguntar "¿cuánto vale `"de"`?" y obtener la respuesta directamente.

2. El diccionario

Un diccionario es una colección de parejas **clave: valor**. Se escribe entre llaves, con dos puntos entre la clave y su valor:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}

print(conteo)
print(conteo["de"])
print(len(conteo))
```

```
{'de': 20, 'que': 9, 'hidalgo': 2}
20
3
```

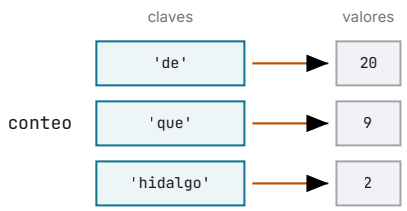


Figura 1. Un diccionario no tiene posiciones. A cada valor se llega por su clave, y `len` cuenta las parejas.

Se indexa con corchetes, igual que una lista, pero lo que va dentro de los corchetes no es una posición sino una **clave**. Pedir una clave que no existe es un error, el equivalente al `IndexError` de las listas:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}
print(conteo["rocín"])
```

```
Traceback (most recent call last):
  File "<stdin>", line 54, in <module>
KeyError: 'rocín'
```

Asignar a una clave hace una de dos cosas. Si la clave no estaba, **agrega** la pareja; si estaba, **reemplaza** su valor. Es la misma instrucción para los dos casos:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}

conteo["rocín"] = 1
```

```
conteo["de"] = 21
conteo["hidalgo"] += 1

print(conteo)
```

```
{'de': 21, 'que': 9, 'hidalgo': 3, 'rocín': 1}
```

Esto es lo que en una lista no se puede hacer: `lista[10] = x` en una lista de tres elementos da `IndexError`, mientras que en un diccionario la clave nueva simplemente aparece.

Para preguntar si una clave está se usa `in`, que en un diccionario mira solo las claves, nunca los valores. Para quitar una pareja se usa `del`:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}

print("hidalgo" in conteo)
print(20 in conteo)

del conteo["que"]
print(conteo)
```

```
True
False
{'de': 20, 'hidalgo': 2}
```

Y un diccionario vacío se escribe `{}`, que es por donde empieza casi todo diccionario que se llena dentro de un ciclo.

Las claves no pueden cambiar

Un diccionario se modifica, igual que una lista: se agregan parejas, se cambian valores y se quitan. Pero las **claves** tienen que ser de un tipo que no se pueda modificar: números, cadenas, tuplas. Una lista no sirve como clave:

```
posiciones = {}
posiciones[[3, 4]] = "reina"
```

```
Traceback (most recent call last):
  File "<stdin>", line 68, in <module>
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

La tupla, en cambio, sí:

```
tablero = {}
tablero[(3, 4)] = "reina"
tablero[(0, 0)] = "torre"

print(tablero[(3, 4)])
print((5, 5) in tablero)
```

```
reina
False
```

Ahí tienes un uso de la tupla que no vimos en la conferencia cuatro: una pareja de coordenadas como clave. El porqué de la regla, y de la palabra unhashable en el error, lo vemos en la sección 7.

3. Recorrer un diccionario

Un `for` sobre un diccionario recorre sus claves, en el orden en que se agregaron:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}

for palabra in conteo:
    print(palabra, conteo[palabra])
```

```
de 20
que 9
hidalgo 2
```

Cuando hacen falta la clave y el valor a la vez, que es casi siempre, el método `items` da las parejas como tuplas, y el `for` las desempaqueta igual que con `enumerate`:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}

for palabra, veces in conteo.items():
    print(f"{palabra:>8} aparece {veces} veces")
```

```
      de aparece 20 veces
     que aparece 9 veces
 hidalgo aparece 2 veces
```

Los métodos `keys` y `values` dan solo las claves o solo los valores. El segundo es el útil, porque los valores no se pueden pedir de otro modo:

```
conteo = {"de": 20, "que": 9, "hidalgo": 2}

print(sum(conteo.values()))
print(max(conteo.values()))
```

```
31
20
```

4. El patrón de conteo

Ya podemos escribir el conteo. Para cada palabra, si ya tiene entrada se le suma uno, y si no la tiene se crea con uno:

```
def frecuencias(palabras):
    """Devuelve un diccionario de cada palabra a las veces que aparece."""
    conteo = {}
```

```
    for palabra in palabras:
        if palabra in conteo:
            conteo[palabra] += 1
        else:
            conteo[palabra] = 1

    return conteo

print(frecuencias("de los de lanza en astillero".split()))
```

```
{'de': 2, 'los': 1, 'lanza': 1, 'en': 1, 'astillero': 1}
```

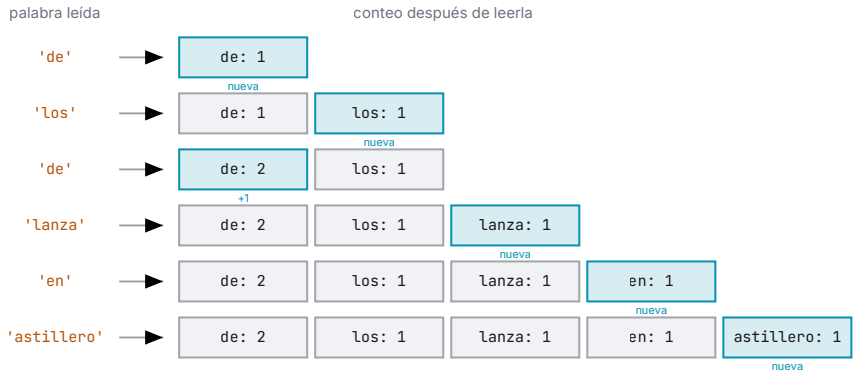


Figura 2. Cada palabra leída toca una sola casilla: la crea si es nueva o le suma uno si ya estaba. El segundo `"de"` no agrega nada, incrementa.

El `if` y el `else` se pueden fundir en una línea con el método `get`, que devuelve el valor de una clave o, si la clave no está, el valor por defecto que le pases en lugar de dar error:

```
conteo = {"de": 2, "los": 1}

print(conteo.get("de", 0))
print(conteo.get("rocín", 0))
```

```
2
0
```

Con `get`, el conteo entero queda así:

```
def frecuencias(palabras):
    """Devuelve un diccionario de cada palabra a las veces que aparece."""
    conteo = {}

    for palabra in palabras:
        conteo[palabra] = conteo.get(palabra, 0) + 1

    return conteo
```

Las dos versiones hacen lo mismo. La primera dice mejor lo que pasa; la segunda es la que vas a ver escrita en todas partes, y conviene reconocerla.

Contra las listas paralelas

Pongamos las dos soluciones a correr sobre el capítulo entero, con el reloj de la conferencia tres:

```
import time

palabras = palabras_de("quijote-cap1.txt")

inicio = time.perf_counter()
vistas, conteos = frecuencias_con_listas(palabras)
con_listas = time.perf_counter() - inicio

inicio = time.perf_counter()
conteo = frecuencias(palabras)
con_diccionario = time.perf_counter() - inicio

print(f"{len(palabras)} palabras, {len(conteo)} distintas")
print(f"con dos listas:     {con_listas * 1000:.2f} ms")
print(f"con un diccionario: {con_diccionario * 1000:.2f} ms")
print(f"el diccionario es {con_listas / con_diccionario:.0f} veces más rápido")
```

```
1895 palabras, 719 distintas
con dos listas:     15.80 ms
con un diccionario: 0.32 ms
el diccionario es 50 veces más rápido
```

El resultado es el mismo y el diccionario es decenas de veces más rápido sobre un solo capítulo. La diferencia crece con el texto, porque con las listas cada palabra recorre una lista de vistas que no para de crecer, y con el diccionario no recorre nada. Por qué no recorre nada es la pregunta de la sección 6.

5. Las diez más comunes

Tener el conteo no es tener la respuesta. Falta encontrar las palabras con más apariciones, y eso es el acumulador de la clase pasada, el que guardaba la palabra más larga vista hasta ahora, cambiando la longitud por el conteo:

```
def mas_comun(conteo):
    """Devuelve la pareja (palabra, veces) con más apariciones."""
    mejor, maximo = "", 0

    for palabra, veces in conteo.items():
        if veces > maximo:
            mejor, maximo = palabra, veces

    return mejor, maximo

print(mas_comun(frecuencias(palabras_de("quijote.txt"))))
```

```
('de', 20)
```

Para las diez primeras basta repetir eso diez veces, sacando del diccionario cada ganador para que la vuelta siguiente encuentre al segundo:

```
def mas_comunes(conteo, k):
    """Devuelve las k parejas (palabra, veces) más frecuentes, de mayor a menor."""
    quedan = dict(conteo)
    resultado = []
```

```
    for vuelta in range(k):
        palabra, veces = mas_comun(quedan)
        resultado.append((palabra, veces))
        del quedan[palabra]

    return resultado

for palabra, veces in mas_comunes(frecuencias(palabras_de("quijote-cap1.txt")), 10):
    print(f"{palabra:>6} {veces:4}")
```

```
    de  122
     y  106
   que   89
     a   44
    el   40
    en   38
    la   35
    su   35
    se   30
   con   27
```

Mira la primera línea de la función. `dict(conteo)` hace una **copia** del diccionario, y es imprescindible: el diccionario se modifica igual que una lista, y si borráramos directamente de `conteo` vaciaríamos el diccionario del que nos llamó. Es la sorpresa de los dos nombres para una misma lista de la conferencia cuatro, ahora con diccionarios.

Este método de buscar el mayor, apartarlo y volver a buscar es una forma de ordenar, y la próxima clase le ponemos nombre y le contamos las operaciones.

El resultado, de todos modos, decepciona: `de`, `y`, `que`, `a`, `el`. Las palabras más frecuentes de cualquier texto en español son las que no dicen nada. Para encontrar las que hablan del capítulo hay que sacar esas primero, y para eso hace falta una colección nueva.

6. Conjuntos

Un **conjunto** es una colección sin repeticiones y sin orden, como los conjuntos de la matemática. Se escribe entre llaves, sin los dos puntos:

```
vocales = {"a", "e", "i", "o", "u"}
dados = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5}

print(len(vocales))
print(dados)
print(4 in dados)
```

```
5
{1, 2, 3, 4, 5, 6, 9}
True
```

El 1, el 3 y el 5 aparecen varias veces en lo que escribimos, y el conjunto los guarda una sola vez. Esa es la operación por la que más se usan: convertir una lista en conjunto elimina los repetidos.

```
palabras = palabras_de("quijote-cap1.txt")
```

```
print(len(palabras), "palabras")
print(len(set(palabras)), "palabras distintas")
```

```
1895 palabras
719 palabras distintas
```

Un conjunto vacío se escribe `set()`, porque `{}` ya estaba tomado por el diccionario vacío. Se le agregan elementos con `add`, y agregar uno que ya estaba no hace nada:

```
vistas = set()
vistas.add("rocín")
vistas.add("hidalgo")
vistas.add("rocín")

print(vistas)
print(type({}))
```

```
{'hidalgo', 'rocín'}
<class 'dict'>
```

Lo que un conjunto no tiene es posiciones. No se puede pedir `vistas[0]`, porque no hay primer elemento, y el orden en que un conjunto imprime sus elementos no tiene por qué ser el orden en que se agregaron:

```
print(vistas[0])
```

```
Traceback (most recent call last):
  File "<stdin>", line 184, in <module>
TypeError: 'set' object is not subscriptable
```

Las operaciones de conjuntos

Los conjuntos traen las operaciones de la matemática, escritas con operadores: `|` para la unión, `&` para la intersección y `-` para la diferencia. Con ellas se comparan vocabularios. Partamos `quijote.txt` en sus tres párrafos, que en el archivo están separados por una línea vacía, y saquemos el vocabulario de cada uno:

```
with open("quijote.txt", encoding="utf-8") as archivo:
    parrafos = archivo.read().split("\n\n")

primero = set(limpiar(parrafos[0]).split())
tercero = set(limpiar(parrafos[2]).split())

print(len(primero), len(tercero))
print(len(primero | tercero), "en alguno de los dos")
print(sorted(primero & tercero))
print(len(primero - tercero), "solo en el primero")
```

```
50 49
92 en alguno de los dos
['de', 'en', 'hidalgo', 'la', 'los', 'que', 'y']
43 solo en el primero
```

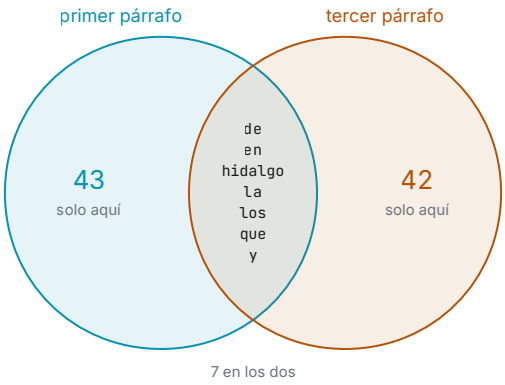


Figura 3. Los vocabularios del primer y el tercer párrafo. Casi todo lo que comparten son palabras que no dicen nada; la única que habla del texto es `hidalgo`.

`sorted` devuelve una lista ordenada, y aquí sirve para que el resultado se imprima en un orden fijo. La intersección es reveladora. Lo que dos párrafos cualesquiera tienen en común son las palabras vacías: artículos, preposiciones, conjunciones. Es la misma lista que encabezaba el conteo.

Sacar las palabras vacías

Así que la solución es un conjunto con las palabras que no queremos contar, y una pregunta con `in` antes de contar cada palabra:

```
VACIAS = {
    "a", "al", "así", "bien", "como", "con", "cual", "cuando", "de", "del",
    "dél", "el", "él", "en", "era", "estaba", "fue", "ha", "había", "la", "las",
    "le", "les", "lo", "los", "más", "mas", "me", "mi", "muy", "ni", "no", "o",
    "para", "pero", "por", "porque", "que", "quien", "se", "ser", "si", "sí",
    "sido", "sin", "su", "sus", "tan", "tenía", "todo", "todos", "un", "una",
    "y", "ya",
}

def frecuencias_sin_vacias(palabras):
    """Como `frecuencias`, pero sin contar las palabras de VACIAS."""
    conteo = {}

    for palabra in palabras:
        if palabra not in VACIAS:
            conteo[palabra] = conteo.get(palabra, 0) + 1

    return conteo

diez = mas_comunes(frecuencias_sin_vacias(palabras_de("quijote-cap1.txt")), 10)

for palabra, veces in diez:
    print(f"{palabra:>9} {veces:4}")
```

```
   nombre   13
caballero   13
      don    6
    razón    6
     vino    6
    rocín    5
  vuestra    5
   amadís    5
```

```
      dio    5
   celada    5
```

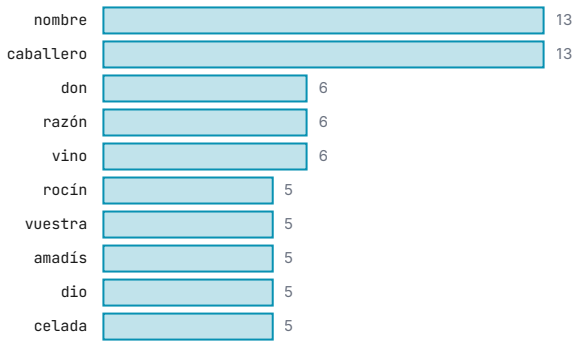


Figura 4. Las diez palabras más frecuentes del primer capítulo, sin las vacías. El capítulo es la historia de cómo un hidalgo se busca un nombre de caballero, y lo mismo hace con su rocín y con su dama.

Ahora sí habla el capítulo: el hidalgo pasa días pensando qué **nombre** ponerse como **caballero** andante, lo mismo hace con su **rocín** y con su dama, y el modelo que tiene en la cabeza es **Amadís** de Gaula. La lista `VACIAS` no es una verdad del idioma sino una decisión: la armamos mirando el resultado y agregando lo que estorbaba. Así se trabaja con datos reales.

Fíjate que `VACIAS` es un conjunto y no una lista. Funcionaría igual con corchetes, pero la pregunta `palabra not in VACIAS` se hace una vez por cada palabra del texto, y en un conjunto esa pregunta es mucho más barata. Cuánto, es lo que sigue.

7. Por qué buscar una clave no cuesta nada

Medimos. La función de abajo pregunta muchas veces si un valor está en una colección y devuelve cuánto tardó cada pregunta. Buscamos el `-1`, que no está, así que la lista tiene que mirar todos sus elementos para poder decir que no:

```python
import time

def segundos_por_busqueda(coleccion, x, veces):
    """Devuelve el tiempo medio que tarda `x in coleccion`."""
    inicio = time.perf_counter()

    for vuelta in range(veces):
        encontrado = x in coleccion

    return (time.perf_counter() - inicio) / veces

medidas = []

for n in [1_000, 10_000, 100_000, 1_000_000]:
    lista = list(range(n))
    conjunto = set(lista)

    en_lista = segundos_por_busqueda(lista, -1, 50)
    en_conjunto = segundos_por_busqueda(conjunto, -1, 100_000)
    medidas.append((n, en_lista, en_conjunto))

    print(f"{n:>9} elementos: lista {en_lista * 1e3:7.3f} ms,"
          f" conjunto {en_conjunto * 1e9:4.0f} ns")
```

```
     1000 elementos: lista   0.015 ms, conjunto   40 ns
    10000 elementos: lista   0.252 ms, conjunto   95 ns
   100000 elementos: lista   2.402 ms, conjunto   79 ns
  1000000 elementos: lista  31.582 ms, conjunto  123 ns
```

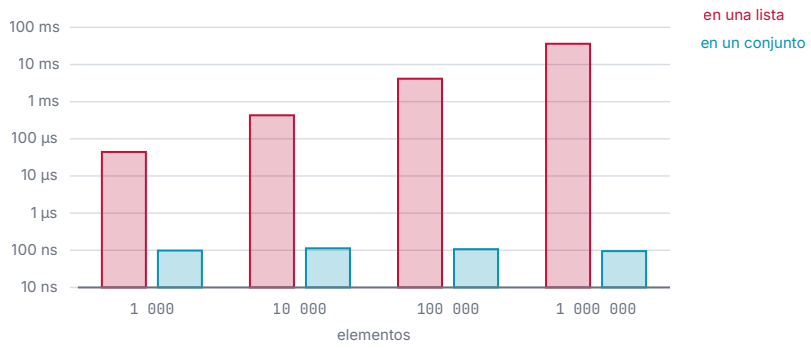


Figura 5. Tiempo de una búsqueda, medido al construir estas notas. La escala es logarítmica: cada raya vale diez veces la de abajo. La lista sube una raya cada vez que el tamaño se multiplica por diez; el conjunto se queda donde estaba.

Un guion bajo dentro de un número, como en `1_000_000`, no cambia su valor; solo lo hace más fácil de leer.

La lista crece en proporción a su tamaño: diez veces más elementos, diez veces más tiempo. El conjunto tarda lo mismo con mil elementos que con un millón. Un diccionario se comporta exactamente igual al buscar una clave, porque por dentro están hechos de la misma manera.

La idea, sin entrar en cómo se implementa, es esta. Una lista guarda sus elementos en fila y, para encontrar uno, no le queda más remedio que mirarlos de uno en uno. Un conjunto o un diccionario, en cambio, calcula **a partir del propio valor** un número que dice en qué casillero guardarlo. Al buscar, hace la misma cuenta, va directo a ese casillero y mira solo ahí. Es como un estante de casilleros con el nombre de cada estudiante, donde no hace falta revisar todos los casilleros para encontrar el tuyo: tu nombre te dice cuál es.

Ese número calculado a partir del valor se llama en inglés hash, y ahí está la razón de la regla de la sección 2. Si una clave pudiera cambiar después de guardarla, su número cambiaría con ella y el diccionario la buscaría en el casillero equivocado. Por eso una lista, que se modifica, es unhashable: no se le puede calcular un casillero fijo. Cómo se calcula ese número y qué pasa cuando dos valores caen en el mismo casillero es materia del segundo semestre.

8. El proyecto completo

El programa pide la ruta del archivo, atiende el error si no existe y reporta las diez palabras más frecuentes, con sus conteos alineados en columna:

```python
def reporte(ruta, k=10):
    """Imprime las k palabras más frecuentes del archivo, sin las vacías."""
    palabras = palabras_de(ruta)
    conteo = frecuencias_sin_vacias(palabras)

    print(f"{len(palabras)} palabras, {len(set(palabras))} distintas")

    for palabra, veces in mas_comunes(conteo, k):
        print(f"  {palabra:<10} {veces:3}")
```

```
reporte("quijote-cap1.txt")
```

```
1895 palabras, 719 distintas
  nombre      13
  caballero   13
  don          6
  razón        6
  vino         6
  rocín        5
  vuestra      5
  amadís       5
  dio          5
  celada       5
```

```
ruta = input("Archivo: ")

try:
    reporte(ruta)
except FileNotFoundError:
    print(f"No encontré el archivo {ruta}")
```

El `<10` dentro del f-string alinea la palabra a la izquierda en diez espacios, y el `3` alinea el número a la derecha en tres. Es lo que convierte una lista de valores en una tabla legible.

El programa tiene un fallo que dejamos como ejercicio. Si pides más palabras de las que el texto tiene distintas, `mas_comun` recibe un diccionario vacío, devuelve la pareja `("", 0)`, y el `del` de la línea siguiente falla.

9. Resumen

- Un diccionario asocia claves con valores. `d[clave]` lee, `d[clave] = valor` agrega o reemplaza, `del d[clave]` quita, y una clave que no está da `KeyError`.
- `in` en un diccionario pregunta por las claves, nunca por los valores.
- `get(clave, defecto)` devuelve el valor o el defecto sin dar error. El patrón de conteo es `conteo[x] = conteo.get(x, 0) + 1`.
- `for clave, valor in d.items()` recorre las parejas, en el orden en que se agregaron.
- Las claves tienen que ser inmutables: números, cadenas o tuplas. Una lista no sirve.
- Un diccionario se modifica como una lista, así que dos nombres pueden compartirlo. `dict(d)` hace una copia.
- Un conjunto guarda elementos sin repetir y sin orden. `set(lista)` quita los repetidos; `set()` es el conjunto vacío, porque `{}` es el diccionario vacío.
- `|`, `&` y `-` son la unión, la intersección y la diferencia.
- Buscar en un conjunto o en un diccionario cuesta lo mismo con diez elementos que con un millón. Buscar en una lista cuesta en proporción a su tamaño.

Ejercicios

1. Escribe `frecuencia_de_letras(ruta)` que devuelva un diccionario de cada letra a las veces que aparece en el archivo, sin contar espacios ni signos. ¿Cuál es la vocal más frecuente en `quijote-cap1.txt`? ¿Y la consonante?

2. Corrige `mas_comunes` para que, si `k` es mayor que el número de palabras distintas, devuelva todas las que hay en lugar de fallar.

3. Escribe `agrupar_por_longitud(palabras)` que devuelva un diccionario de cada longitud a la lista de palabras distintas de esa longitud. Por ejemplo, la clave `11` del primer capítulo debe incluir `entresemana`.

4. Dos palabras son anagramas si tienen las mismas letras con las mismas repeticiones, como `roma` y `amor`. Escribe `son_anagramas(a, b)` comparando dos diccionarios de conteo de letras. Después escribe `grupos_de_anagramas(palabras)` usando como clave `""".join(sorted(palabra))`, y busca los grupos de más de una palabra en el capítulo.

5. Escribe `invertir(conteo)` que reciba un diccionario de palabras a conteos y devuelva uno de cada conteo a la lista de palabras con ese conteo. ¿Cuántas palabras del capítulo aparecen una sola vez?

6. Tienes la lista de los estudiantes de un grupo y las listas de los que aprobaron el primer y el segundo examen. Con operaciones de conjuntos, di quiénes aprobaron los dos, quiénes aprobaron exactamente uno y quiénes no aprobaron ninguno.

7. Escribe `traducir(frase, diccionario)` que traduzca palabra a palabra con un diccionario como `{"hidalgo": "nobleman", "rocín": "nag"}`, dejando igual las palabras que no estén. Resuélvelo con `get` y sin ningún `if`.

8. Sin ejecutarlos, di qué imprime cada uno de estos tres programas. Después compruébalo.

   (a)

   ```
   d = {"a": 1, "b": 2}
   d["a"] = d.get("c", 10) + d["b"]
   print(d)
   ```

   (b)

   ```
   print(len({1, 2, 2, 3, 3, 3}))
   print(len({"uno": 1, "uno": 2}))
   ```

   (c)

   ```
   a = {"x": 1}
   b = a
   b["x"] = 2
   print(a)
   ```